QUEEN'S UNIVERSITY FINAL EXAMINATION
FACULTY OF ARTS AND SCIENCE
SCHOOL OF COMPUTING

CISC 220 001 – Professor Burton Ma
April 23, 2022

INSTRUCTIONS TO STUDENTS:
This examination is 3 HOURS in length.
There are 6 questions worth a total of 80 marks in this examination.
Please answer all questions in the answer booklets.
No aids are allowed.

Put your student number on all pages of all answer booklets, including the front.
GOOD LUCK!

PLEASE NOTE:
Proctors are unable to respond to queries about the interpretation of exam questions.
Do your best to answer exam questions as written.

This material is copyrighted and is for the sole use of students registered in CISC 220 and writing this exam. This material shall not be distributed or disseminated. Failure to abide by these conditions is a breach of copyright and may also constitute a breach of academic integrity under the University Senate's Academic Integrity Policy Statement.

Question 1(a) (3 points):
Write a Bash script that prints the sequence of integer values from 1 to 19 in steps of 2 (1, 3, 5, 7, …, 19) to standard output, one value per line, using a loop.

Question 1(b) (3 points):
Consider the following Bash script:

```
#!/bin/bash
arr=($(cat "$1"))
```

The script as it is shown above expects one command line argument that is the name of a file containing some integer values, one value per line. It stores the integer values in the array `arr`.

Complete the script so that it computes the sum of the values in the array `arr` using a loop and prints the sum to standard output.

You do not need to perform any error checking.

Question 1(c) (4 points):
Write a Bash script that changes the permissions of all the regular files (but not the directories) in the current directory (but not files in any subdirectory) so that the user has only read permission on the files and all other users have no permissions on the files. For each file where the user did not originally have permission to read the file, your script should print to standard output the message:

`Adding read permission to` *`filename`*

where *`filename`* is the name of the file whose permissions were changed.

Your script does not need to consider hidden files.

Question 2(a) (3 points):
Consider the following C function that searches an array `arr` of positive values for the maximum value in the array and returns the index of the maximum value; the array is assumed to have at least two values and the last value in array is assumed to be `-1`:

```
size_t maxidx(const int *arr) {
    size_t index = 0;
    int hi = arr[index];
    for (size_t i = 1; arr[i] != -1; i++) {
        if (arr[i] > hi) {
            index = i;
            hi = arr[i];
        }
    }
    return index;
}
```

Instead of returning an index, how would you change the `return` statement so that the function returned a pointer to the maximum value instead of returning an index (assume that the return type is changed to `int *`)? Your answer should change only the `return` statement; do not modify any other part of the function.

Question 2(b) (7 points):
Write a function named `ptr2max` that when called as shown below:

```
// a is an array of int whose last value is -1
size_t i = maxidx(a);
int *p = ptr2max(a);
```

returns a pointer `p` such that `p` points to the maximum value in `a`.

Your function must not use array indexing (this means that you cannot use square brackets `[ ]` in your answer), and it must not call another function. It should use a pointer (or pointers), pointer dereferencing, and pointer arithmetic instead of array indexing. Note that `ptr2max` does not receive the length of the array as input.

Question 3(a) (4 points):
Show how to use the `malloc` function to:

i. allocate memory for a single `int` value
ii. allocate memory for an array of 10 `float` values

Question 3(b) (4 points):
Show how to use the `malloc` function to allocate memory for an array of 5 strings where each string has length 10 (hint: there should be six calls to `malloc`, one for the array and one for each string)

Question 3(c) (2 points):
Show how to use the `realloc` function so that the first string in your array from Question 3(b) can safely store a null-terminated string containing 20 characters.

Question 4(a) (4 points):
An assessment item for a course is an item that receives a grade (e.g., a quiz, assignment, exam, etc). Every assessment item has a name, a maximum grade, and a weight (the percentage that the item contributes to the course grade).

Define a C `struct` that can store the information for an assessment item. Assume that the name is a string less than 128 characters in length, the maximum grade is an integer value, and the weight is a floating-point value.

Question 4(b) (2 points):
Provide a C `typedef` for the struct in Question 4(a).

Question 4(c) (4 points):
Write a C function `copy_item` that takes an assessment item as a parameter and returns an independent copy of the item. For example, suppose your typedef from Question 4(b) is called `my_item`, then:

```
my_item assignment2 = copy_item(assignment1);
```

would allow a programmer to change the name, maximum grade, and weight of `assignment2` without affecting `assignment1`.

Question 5(a) (2 points):
Consider the generic stack data structure (that we implemented in the course) where the function `stack_push(void *elem)` pushes an element pointed to by `elem` onto the stack. Suppose that a programmer writes the following function:

```
void push_zero(stack *s) {
    double z = 0.0;
    stack_push(s, &z);
}
```

After calling the function, is the top element of the stack a zero? Explain why or why not.

Question 5(b) (2 points):
The following function declaration:

```
bool is_zero(const void *e);
```

declares a function that returns true if the element that `e` points to is equal to zero. Assume that you have a definition of the `is_zero` function in a C source code file. Create a variable whose type is a pointer to a function and initialize its value so that it points at the `is_zero` function. Your variable must not be a void pointer.

Question 5(c) (2 points):
Provide an implementation of the `is_zero` function where the value pointed to by `e` has type `double`. Note that the function parameter type must be `const void *`.

Question 5(d) (4 points)
The stack data structure that we implemented in the course used a sequence of linked nodes to store the elements. The node and stack typedefs are shown below:

```
typedef struct node node;
struct node {
    void *elem;
    node *next;
};
typedef struct stack stack;
struct stack {
    node *top;
    size_t size;
};
```

Provide an implementation of a function that tests if a generic stack contains multiple elements equal to zero (i.e., the stack has two or more elements equal to zero). The inputs to the function are a pointer to a `stack` and a pointer to a function having the same declaration as `is_zero` from Part (b). The return type of the function is `bool`.

Question 6(a) (10 points):
Write the following C function:

```
int read_strings(const char* filename, char *s[static 100])
```

that opens the file having name `filename`, reads the contents of the file, and then closes the file. Assume that the file exists, is readable, and consists of an unspecified number of lines where each line is a string. The function should read each line of the file and store each line in the array `s` (the contents of the first line of the file should be stored in `s[0]`, the second line in `s[1]`, and so on). You should assume that `s[i]` has sufficient capacity to store the contents of line `i` (i.e., do not dynamically allocate memory in this question). The function should return the number of lines read from the file. If there are more than 100 lines in the file, then the function should store only the first 100 lines but still return the actual number of lines in the file.

Question 6(b) (10 points):
Write the following C function:

```
void shortest_to_front(int n, char *s[static n])
```

that finds the shortest string in the array `s` and moves the shortest string to the front of the array by swapping the shortest string with the string at the front of the array. For example, if `s` is the array `["zebra", "anteater", "bat", "frog"]` then calling the function would modify `s` so that it becomes the array `["bat", "anteater", "zebra", "frog"]`.

Question 6(c) (2 points):
Write the following C function:

```
void sort_by_length(int n, char *s[static n])
```

that sorts the array of strings `s` in increasing string length using the `shortest_to_front` function from Question 6(b). For example, if `s` is the array `["zebra", "anteater", "bat", "frog"]` then calling the function would modify `s` so that it becomes the array `["bat", "frog", "zebra", "anteater"]`. To move elements of the array `s`, you must use the `shortest_to_front` function (this means that you cannot write something like `s[i] = …` to assign an array element).

Hint: Call `shortest_to_front` (`n-1`) times. You don't need to answer 6(b) to use `shortest_to_front` in your answer.

Question 6(d) (8 points):
Write the main function of a C program that requires one command-line argument which is a file name. If one command-line argument is not given, then the function should exit with an exit status of 1.

The input file is a plain-text file made up of individual lines of text. The program should read the lines of text into an array of strings (using the `read_strings` function). The program should then sort the lines of the file by increasing string length (using the `sort_by_length` function). Finally, the program should print the lines of the file from shortest line to longest line, each line as a separate line of output. You may assume that each line of text in the input file is less than 100 characters in length.